Lecture 17: Dictionaries (Chapter 10)

유준수 (Joon Soo Yoo, Ph.D.)

2026년 4월 9일

방산첨단보안연구실 (DAS Lab)
전북대학교 첨단방위산업학과

10.1 Dictionary Basics

데이터의 이름표: 딕셔너리의 구조와 특징

딕셔너리(Dictionary)란?

딕셔너리는 키(Key)와 값(Value)의 쌍으로 이루어진 컨테이너 객체입니다.

- **구조:** `{Key1: Value1, Key2: Value2, ...}`
- **비유:** 영어 사전에서 '단어(Key)'를 찾으면 그 '뜻(Value)'이 나오는 것과 같습니다.

예시: 요일 데이터 (days)

```
days = {"Sunday": 1, "Monday": 2, "Tuesday": 3}
```

Keys: "Sunday", "Monday", "Tuesday" (데이터를 찾는 이름표)

Values: 1, 2, 3 (실제 담긴 데이터)

딕셔너리의 타입과 확인

파이썬에서 딕셔너리는 dict라는 타입으로 구현됩니다.

타입 확인 예제

```
encoding = {"a": 97, "b": 98, "A": 65, "B": 66}
print(type(encoding))
```

결과 → <class 'dict'>

알아두기

type() 함수를 통해 해당 변수가 딕셔너리인지 리스트인지 명확히 구분할 수 있습니다.

중요 속성: 키(Key)는 중복될 수 없다!

딕셔너리에서 키는 데이터를 찾는 유일한 '주소'와 같기 때문에 중복을 허용하지 않습니다.

1. 키는 유일해야 함

같은 이름의 키가 두 개 있을 수 없습니다. (나중에 쓴 값이 이전 값을 덮어씀)

2. 값은 중복 가능

서로 다른 키가 같은 값을 가질 수는 있습니다.
`{"Lee": 100, "Kim": 100}` (O)

대소문자 구분

`"a"`와 `"A"`는 서로 다른 문자로 인식되므로, `{"a": 97, "A": 65}`는 중복 키가 아닙니다.

Task: 스마트폰 연락처 리스트 만들기

스마트폰에서 '친구 이름'을 누르면 '전화번호'가 연결되는 딕셔너리를 만들어 봅시다.

Mission: 연락처 저장 및 중복 테스트

1. **연락처 생성:** 친구의 별명을 **Key**로, 전화번호를 **Value**로 하는 딕셔너리 `my_contacts`를 만드세요.
2. **데이터 예시:** `{"짱구": "010-1234-5678", "철수": "010-9876-5432"}`
3. **타입 확인:** `type()` 함수를 사용하여 이 객체가 정말 `dict` 타입인지 확인해 보세요.
4. **중복 테스트:** 이미 존재하는 이름("짱구")으로 다른 번호를 한 번 더 저장해 보세요. 기존 번호가 어떻게 변하나요?

10.2 Dictionary Creation

빈 상자부터 채워진 상자까지: 딕셔너리 생성법

방법 1: 빈 딕셔너리 만들기

데이터를 담기 전, 빈 주머니를 만드는 두 가지 방법이 있습니다.

중괄호 {} 사용 (가장 일반적)

```
dict_1 = {}
```

가장 가볍고 간결하여 실무에서 가장 자주 쓰이는 방식입니다.

dict() 함수 사용

```
dict_2 = dict()
```

객체 지향적인 관점에서 명시적으로 딕셔너리를 생성할 때 활용합니다.

방법 2: 중괄호로 초기값 설정하기

처음부터 데이터를 채워서 만들 때는 **콜론(:)**과 **쉼표(,)**의 위치가 중요합니다.

오늘의 점심 메뉴판 (Menu)

```
menu = {"pizza": 15000, "pasta": 12000, "drink": 2000}
```

- **구분:** 키(Key)와 값(Value) 사이에는 **콜론(:)**을 찍습니다.
- **나열:** 각 아이템 사이에는 **쉼표(,)**를 찍습니다.

방법 3: dict() 함수로 멋지게 만들기 (1)

기존에 있는 데이터(리스트 등)를 딕셔너리로 변환하거나 복사할 때 유용합니다.

튜플 리스트로부터 생성

```
my_list = [("apple", 2), ("banana", 3)]
my_dict = dict(my_list)
 → {'apple': 2, 'banana': 3}
```

다른 딕셔너리 복사

```
old_dict = {"a": 1, "b": 2}
new_dict = dict(old_dict)
```

→ 내용물은 같지만 원본과는 별개인 새로운 딕셔너리 객체가 만들어집니다.

방법 3: dict() 함수로 멋지게 만들기 (2)

키워드 인자(Keyword Arguments)를 사용하면 따옴표 없이도 깔끔하게 작성이 가능합니다.

키워드 인자 활용 예시

```
fruit_stock = dict(apple=10, orange=5, grape=20)
```

주의사항

이 방식을 쓸 때는 키 이름에 따옴표("")를 붙이지 않습니다. 파이썬이 자동으로 문자열 키로 변환해 줍니다.

Task: 나를 소개하는 데이터 상자

자신의 정보를 담은 프로필 딕셔너리를 만들어 봅시다.

Mission: 자기소개 딕셔너리 생성

1. **딕셔너리 이름:** `my_info`
2. **포함할 데이터(Key):** "first_name", "last_name", "age"
3. **방법 선택: 중괄호({})** 방식이나 **dict()** 함수 방식 중 하나를 골라 본인의 정보를 입력하세요.
4. **출력:** 완성된 `my_info`를 출력하여 데이터가 잘 담겼는지 확인합니다.

10.3 Dictionary Operations

데이터 꺼내기, 바꾸기, 그리고 버리기

데이터 접근: 대괄호 vs get()

딕셔너리에서 값을 가져올 때는 '안전함'의 정도에 따라 두 가지 방법을 씁니다.

1. 대괄호 [] (직구)

```
my_dict = {"apple": 2, "banana": 3}
print(my_dict["apple"])   # 2
```

! 없는 키를 넣으면 에러(Exception)가 발생하며 프로그램이 멈춥니다.

2. get() 메서드 (안전장치)

```
print(my_dict.get("apple"))   # 2
print(my_dict.get("melon"))   # None (에러 없이 None 반환)
print(my_dict.get("melon", 0))   # 0 (기본값 설정 가능)
```

데이터 목록 보기: Keys, Values, Items

딕셔너리의 전체 구성을 파악하고 싶을 때 사용합니다.

세 가지 뷰(View) 객체

```
nums = {"one": 1, "two": 2}
```

- `nums.keys()`: 모든 키 목록 → `dict_keys(['one', 'two'])`
- `nums.values()`: 모든 값 목록 → `dict_values([1, 2])`
- `nums.items()`: 키-값 쌍 목록 → `dict_items([('one', 1), ('two', 2)])`

팁

결과를 리스트처럼 쓰고 싶다면 `list()` 함수로 감싸주세요!

```
list(nums.keys())    # ['one', 'two']
```

데이터 수정 및 추가: 딕셔너리는 가변적(Mutable)

딕셔너리는 생성 후에도 자유롭게 내용을 바꿀 수 있습니다.

방법 1: 대괄호 []

```
food = {"Soup": "$15"}
food["Soup"] = "$11"      # 기존 값 수정
food["Sushi"] = "$16"    # 새로운 항목 추가
```

방법 2: update() 메서드

```
food.update({"Kabob": "$20", "Sushi": "$18"})
```

여러 항목을 한꺼번에 수정하거나 추가할 때 매우 편리합니다.

데이터 삭제: del과 pop()

필요 없는 데이터를 지우는 두 가지 깔끔한 방법입니다.

del 키워드 (그냥 지우기)

```
del food["Soup"]   # "Soup" 항목이 영원히 삭제됩니다.
```

pop() 메서드 (꺼내고 지우기)

```
val = food.pop("Kabob")   # 지우면서 그 값을 변수에 담아줍니다.
print(val)   # "$20"
```

Task: 나만의 중고차 재고 관리

딕셔너리를 사용하여 자동차 재고 리스트를 관리해 봅시다.

Mission: 자동차 데이터 업데이트

1. **초기화:** 빈 딕셔너리 `cars`를 만드세요.
2. **추가:** `"Mustang": 10, "Volt": 3` 데이터를 추가하세요.
3. **수정:** `"Mustang"`의 수량을 2로 변경하세요.
4. **삭제:** `"Volt"` 항목을 딕셔너리에서 지우세요.
5. **확인:** 최종 딕셔너리 내용을 출력해 보세요.

팁

실제 서버에서 사용자 데이터를 지울 때는 실수 방지를 위해 `del`보다 `pop()`을 써서 확인 절차를 거치는 경우가 많음

10.4 Conditionals and Looping

딕셔너리 검색과 순회: 데이터 뒤져보기

딕셔너리 조건문: 존재 여부 확인하기

특정 데이터가 있는지 없는지 확인할 때는 in 연산자를 사용합니다.

1. 키(Key) 존재 확인

```
movies = {"The Godfather": 1974, "Interstellar": 2014}
print("Interstellar" in movies)   # True
```

2. 값(Value) 존재 확인

```
print(2014 in movies.values())   # True
# 주의: 그냥 'in movies'는 키만 검사하지만, '.values()'는 값을 검사합니다.
```

반복문 활용 (1): 키와 값을 동시에 꺼내기

딕셔너리의 데이터를 하나씩 처리할 때 가장 많이 쓰이는 '국룰' 패턴입니다.

.items()를 활용한 순회

```
zip_codes = {"Berkeley": 94709, "Santa Cruz": 95064}
for city, code in zip_codes.items():
    print(f"{city}의 우편번호는 {code}입니다.")
```

팁

변수 이름을 key, value 대신 city, code처럼 의미 있게 지으면 코드가 훨씬 읽기 좋아짐!

반복문 활용 (2): 키 또는 값만 꺼내기

필요한 정보만 골라서 루프를 돌릴 수도 있습니다.

키(Key)만 꺼내기

```
for name in fruit_count.keys():   # 혹은 그냥 'for name in fruit_count:'
    print(name)
```

값(Value)만 꺼내기 (예: 합계 계산)

```
total = 0
for count in fruit_count.values():
    total += count   # 개수만 다 더하고 싶을 때 유용!
```

Task: 우리 과 동아리 회비 정산

동아리원들의 회비 납부 현황을 확인하는 프로그램을 만들어 봅시다.

Mission: 미납자 명단 뽑기

1. **데이터 생성:** 이름(Key)과 납부액(Value)이 담긴 `dues` 딕셔너리를 만드세요.
   `{"짱구": 5000, "철수": 0, "유리": 5000, "훈이": 0}`
2. **반복문 작성:** `.items()`를 사용하여 딕셔너리를 순회하세요.
3. **조건문 추가:** 납부액이 **0**인 사람의 이름을 찾아 "OO아, 회비 내야지!"라고 출력하세요.
4. **심화:** 총 걷힌 회비의 합계를 구해 출력해 보세요.

팁

실제 서비스에서는 이런 식으로 미납자에게 자동으로 알림 톡을 보내는 기능을 구현할 수 있음.

Any Questions?